P0452r1
Unifying <numeric> Parallel Algorithms

Published Proposal, 1 March 2017

This version:
http://wg21.link/P0452r1

Author:
Bryce Adelstein Lelbach (Lawrence Berkeley National Laboratory)

Audience:
SG1, LEWG, LWG

Toggle Diffs:
☐ Hide deleted text

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Table of Contents

1 Overview
1.1 `inner_product` Cannot Be Parallelized
1.2 `inner_product` is a Form of `transform_reduce`
1.3 `transform_reduce` is Missing an Overload
1.4 `transform_reduce` Parameter Order is Odd
1.5 `transform_inclusive_scan` and `transform_exclusive_scan` Parameter Order is Odd

2 Proposed Resolution

3 FAQ

4 Wording
4.1 Wording for `transform_reduce`/`inner_product`
4.2 Wording for `transform_exclusive_scan`
4.3 Wording for `transform_inclusive_scan`

5 Acknowledgement

References
Informative References

§ 1. Overview

US 161 and US 184 identify issues relating to the `ExecutionPolicy` (e.g. parallel) signatures of the four pre-C++17 sequence algorithms (`accumulate`, `inner_product`, `partial_sum` and `adjacent_difference`) found in `<numeric>`.

All of these issues revolve around the strength of the ordering requirements of these algorithms and the new `<numeric>` algorithms with weaker ordering guarantees that were introduced in the Parallelism TS v1 and C++17 (`reduce`, `transform_reduce`, `inclusive_scan`, `exclusive_scan`, `transform_inclusive_scan` and `transform_exclusive_scan`).

This paper proposes two types of changes:

- Addition/removal of `<numeric>` sequence algorithm signatures.
- Changes to the order of parameters of `<numeric>` sequence algorithm signatures.

§ 1.1. `inner_product` Cannot Be Parallelized

Currently, `accumulate`, `inner_product` and `partial_sum` use "accumulator-style" wording:

> **26.8.2 Accumulate [accumulate]**:
>
> ```
> template <class InputIterator,
>           class T>
>   T accumulate(InputIterator first, InputIterator last,
>                T init);
> template <class InputIterator,
>           class T,
>           class BinaryOperation>
>   T accumulate(InputIterator first, InputIterator last,
>                T init,
>                BinaryOperation binary_op);
> ```
>
> 1 *Effects:* Computes its result by initializing the accumulator `acc` with the initial value `init` and then modifies it with `acc = acc + *i` or `acc = binary_op(acc, *i)` for every iterator `i` in the range `[first, last)` in order.
>
> 2 *Requires:* `T` shall meet the requirements of `CopyConstructible` (Table 22) and `CopyAssignable` (Table 24) types. In the range `[first, last]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.

> **26.8.5 Inner product [inner.product]**:
>
> ```
> template <class InputIterator1, class InputIterator2,
>           class T>
>   T inner_product(InputIterator1 first1, InputIterator1 last1,
>                   InputIterator2 first2,
>                   T init);
> template <class InputIterator1, class InputIterator2,
>           class T,
>           class BinaryOperation1,
>           class BinaryOperation2>
>   T inner_product(InputIterator1 first1, InputIterator1 last1,
>                   InputIterator2 first2,
>                   T init,
>                   BinaryOperation1 binary_op1,
>                   BinaryOperation2 binary_op2);
> ```
>
> 1 *Effects:* Computes its result by initializing the accumulator `acc` with the initial value `init` and then modifying it with `acc = acc + (*i1) * (*i2)` or `acc = binary_op1(acc, binary_op2(*i1, *i2))` for every iterator `i1` in the range `[first1, last1)` and iterator `i2` in the range `[first2, first2 + (last1 - first1))` in order.
>
> 2 *Requires:* `T` shall meet the requirements of `CopyConstructible` (Table 22) and `CopyAssignable` (Table 24) types. In the ranges `[first1, last1]` and `[first2, first2 + (last1 - first1)]` `binary_op1` and `binary_op2` shall neither modify elements nor invalidate iterators or subranges.

26.8.6 Partial sum [partial.sum]

```
template <class InputIterator, class OutputIterator>
  OutputIterator partial_sum(InputIterator first, InputIterator last,
                             OutputIterator result);
template <class InputIterator, class OutputIterator,
          class BinaryOperation>
  OutputIterator partial_sum(InputIterator first, InputIterator last,
                             OutputIterator result,
                             BinaryOperation binary_op);
```

1 *Effects:* For a non-empty range, the function creates an accumulator `acc` whose type is `InputIterator`'s value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1, last)` in order, `acc` is then modified by `acc = acc + *i` or `acc = binary_op(acc, *i)` and the result is assigned to `*(result + (i - first))`.

2 *Returns:* `result + (last - first)`.

3 *Complexity:* Exactly `(last - first) - 1` applications of the binary operation.

4 *Requires:* `InputIterator`'s value type shall be constructible from the type of `*first`. The result of the expression `acc + *i` or `binary_op(acc, *i)` shall be implicitly convertible to `InputIterator`'s value type. `acc` shall be writable (24.2.1) to the `result` output iterator. In the ranges `[first, last]` and `[result, result + (last - first)]` `binary_op` shall neither modify elements nor invalidate iterators or subranges.

5 *Remarks:* `result` may be equal to `first`.

This wording specifies that these algorithms perform the following operations:

- Create an accumulator `acc` which is initialized with `init`.
- For (the) iterator(s) in the range(s) **in order**, modify `acc` by applying an update operation which takes `acc` and the dereferenced value(s) of the iterator(s) as arguments.

The ordering requirement is necessary to ensure that these algorithms are well-defined for **non-associative** and/or **non-commutative** operations, such as floating-point addition and multiplication (non-associative and commutative), and subtraction (non-associative and non-commutative).

Currently, the wording forces each iteration to depend on the prior iteration to ensure the correct ordering of accumulation. This introduces a loop-carried dependency that makes it impossible to parallelize.

To parallelize `accumulate` and `inner_product` we need to be able to re-order applications of the operations, partition the workload into sub-tasks and then combine the results of the sub-tasks together using the operator. This, however, would give a non-deterministic result for non-associative or non-commutative operations.

For `partial_sum`, re-ordering is not possible due to the nature of the algorithm. However, partitioning is possible. Thus, you get a non-deterministic result for non-associative operations but it is fine for non-commutative operations.

To allow parallelization of `accumulate` and `partial_sum` without relaxing the existing ordering, new algorithms with weaker ordering were introduced instead of adding `ExecutionPolicy` signatures of the existing ones: `reduce` (a weaker `accumulate`) and `inclusive_scan` (a weaker `partial_sum`).

However, for `inner_product`, no new algorithm with weaker ordering was introduced and an `ExecutionPolicy` signature **was** added instead. US 184 points out that this signature is not useful as it cannot be parallelized due to the ordering constraints.

§ 1.2. `inner_product` is a Form of `transform_reduce`

Just as `reduce` is the parallelizable counterpart of `accumulate` (same basic operation but without a specific ordering), `inner_product` has a natural counter-part. Consider this possible implementation of `inner_product`:

```
template <class InputIt1, class InputIt2,
          class T,
          class ReductionOperation, class BinaryTransformOperation>
T inner_product(InputIt1 first1, InputIt1 last1,
                InputIt2 first2, T init,
                ReductionOperation reduce_op,
                BinaryTransformOperation transform_op)
{
    while (first1 != last1)
    {
        init = reduce_op(init, transform_op(*first1, *first2));
        ++first1;
        ++first2;
    }
    return value;
}
```

The application of `transform_op` to the sequence is a binary `transform`:

```
template <class InputIt1, class InputIt2,
          class OutputIt, class BinaryTransformOperation>
OutputIt transform(InputIt1 first1, InputIt1 last1, InputIt2 first2,
                   OutputIt o_first, BinaryTransformOperation transform_op)
{
    while (first1 != last1)
    {
        *o_first++ = transform_op(*first1++, *first2++);
    }
    return o_first;
}
```

And the application of `reduce_op` accumulates the transformed values:

```
template <class InputIt, class T, class ReductionOperation>
T accumulate(InputIt first, InputIt last, T init,
             ReductionOperation reduce_op)
{
    for (; first != last; ++first)
    {
        init = reduce_op(init, *first);
    }
    return init;
}
```

So, `inner_product` is an ~~`accumulate`~~s `reduce`s of a `transform`ed sequence. It's a `transform_reduce`!.

§ 1.3. `transform_reduce` is Missing an Overload

However, `transform_reduce` is missing an signature for a binary transform; only signatures with unary transform operations are currently specified by the C++17 CD. I call this missing signature **binary-binary `transform_reduce`**, since both the reduction and transformation operation are binary operations.

This signature, which is equivalent to parallel `inner_product` with weaker ordering, is very useful. Many of the typical examples of `transform_reduce` usage that I've seen use tricks to perform a binary-binary `transform_reduce` using the unary-binary `transform_reduce` that is in the C++17 CD.

The typical `transform_reduce` dot product example (similar to what is found in the original `transform_reduce` proposal [N3960]) looks like this:

```
std::vector<std::tuple<double, double> > XY = // ...

double dot_product = std::transform_reduce(
    std::par_unseq,

    // Input sequence.
    XY.begin(), XY.end(),

    // Unary transformation operation.
    [](std::tuple<double, double> const& xy)
    {
      // Array of struct means this is tricky to execute on vector hardware:
      //
      // memory layout: X[0] Y[0] X[1] Y[1] X[2] Y[2] X[3] Y[3] ...
      //
      // op #0: load a pack of Xs (they aren't contiguous; the load will be
      //                           strided, may need to access multiple
      //                           cache lines and may be harder for the
      //                           hardware prefetcher to handle)
      // op #1: load a pack of Ys (same as above)
      // op #2: multiply the pack of Xs by vector of Ys
      return std::get<0>(xy) * std::get(xy);
    },

    double(0.0), // Initial value for reduction.

    // Binary reduction operation.
    std::plus<double>()
  );
```

Note that this is array-of-structs NOT struct-of-arrays. The HPX and THRUST dot product examples use iterator tricks (zip iterators or counting iterators) to switch to a struct-of-arrays scheme. The zip iterator trick is shown below, using Boost:

```
std::vector<double> X = // ...
std::vector<double> Y = // ...

double dot_product = std::transform_reduce(
    std::par_unseq,

    // Input sequence.
    boost::make_zip_iterator(X.begin(), Y.begin()),
    boost::make_zip_iterator(X.end(), Y.end()),

    // Unary transformation operation.
    [](auto&& xy) // std::tuple<double, double>-esque
    {
        // Struct of arrays means this is easier to execute on vector hardware:
        //
        // memory layout: X[0] X[1] X[2] X[3] ... Y[0] Y[1] Y[2] Y[3] ...
        //
        // op #0: load a pack of Xs (elements are contiguous, load will access
        //                           only one cache line and the hardware
        //                           prefetcher will easily track the memory
        //                           stream)
        // op #1: load a pack of Ys (same as above)
        // op #2: multiply the pack of Xs by the pack of Ys
        return std::get<0>(xy) * std::get<1>(xy);
    },

    double(0.0), // Initial value for reduction.

    // Binary reduction operation.
    std::plus<double>()
);
```

More examples of this pattern can be found in HPX (zip iterator example and counting iterator example) and THRUST (zip iterator example).

1.4. `transform_reduce` Parameter Order is Odd

The missing binary-binary `transform_reduce` would look like this:

```
template <class InputIt1, class InputIt2,
          class BinaryTransformOperation,
          class T,
          class ReductionOperation> // Always binary.
T transform_reduce(InputIt1 first1, InputIt1 last1,
                   InputIt2 first2,
                   BinaryTransformOperation transform_op,
                   T init,
                   ReductionOperation reduce_op);
```

Note that the order of parameters, which is consistent with the other `transform_reduce` signatures, is inconsistent with `inner_product`, `transform` and `accumulate`:

```
template <class InputIt1,
          class UnaryTransformOperation,
          class T,
          class ReductionOperation> // Always binary.
T transform_reduce(InputIt1 first1, InputIt1 last1,
                   UnaryTransformOperation transform_op,
                   T init,
                   ReductionOperation reduce_op);

template <class InputIt1, class InputIt2,
          class T>
T inner_product(InputIt1 first1, InputIt1 last1, InputIt2 first2,
                T init);

template <class InputIt1, class InputIt2,
          class T,
          class ReductionOperation, // Always binary.
          class BinaryTransformOperation>
T inner_product(InputIt1 first1, InputIt1 last1, InputIt2 first2,
                T init,
                ReductionOperation reduce_op,
                BinaryTransformOperation transform_op);


template <class InputIt1, class InputIt2, class OutputIt,
          class BinaryTransformOperation>
OutputIt transform(InputIt1 first1, InputIt1 last1, InputIt2 first2,
                   OutputIt o_first,
                   BinaryTransformOperation transform_op);

template <class InputIt1,
          class T,
          class ReductionOperation>
T accumulate(InputIt1 first1, InputIt1 last1,
             T init,
             ReductionOperation reduce_op);
```

The inconsistencies are:

- In `inner_product`, `accumulate` and `reduce` (which is not shown but has the same interface as `accumulate`), the `init` parameter comes after the iterator parameters and before the operation parameters. In `transform_reduce`, it is in between the transformation operation parameter and the reduction operation parameter.

- In `inner_product`, the reduction operation parameter comes before the transformation operation parameter. In `transform_reduce`, the reduction operation parameter comes before the transformation operation parameter.

1.5. `transform_inclusive_scan` and `transform_exclusive_scan` Parameter Order is Odd

Like `transform_reduce`, the new `transform_inclusive_scan` and `transform_exclusive_scan` algorithms have parameter orders that are inconsistent with existing algorithms that they are closely related to, like `accumulate`, `partial_sum` and `inner_product`. The specific issues:

- In `exclusive_scan`, the `init` parameter comes after the iterator parameters and before the operation parameters. In `transform_exclusive_scan`, the `init` parameter is after the transformation operation parameter and the reduction operation parameter.
- In `inclusive_scan`, the reduction operation parameter comes before the transformation operation parameter. In `transform_inclusive_scan` and `transform_exclusive_scan`, the reduction operation parameter comes before the transformation operation parameter.

2. Proposed Resolution

The following changes would resolve the issues with the `<numeric>` raised by US 161 and US 184:

- Remove the `ExecutionPolicy` signature for `inner_product`.
- Add binary-binary `transform_reduce` signatures (four signatures: with and without `ExecutionPolicy` parameters, and with and without defaulted operations).
- Change the parameter order for all `transform_reduce` signatures so that:
  - The initial value parameter comes after the iterator parameters and before the operation parameters.
  - The reduction operation parameter comes before the transformation operation parameters.
- Change the parameter order for `transform_*_scan` signatures so that:
  - For `transform_exclusive_scan`, the initial value parameter comes after the iterator parameters and before the operation parameters.
  - For `transform_inclusive_scan` and `transform_exclusive_scan`, the reduction operation parameter comes before the transformation operation parameters.

With these changes, a parallel dot product over struct-of-arrays data can be written as:

```
std::vector<double> X = // ...
std::vector<double> Y = // ...

double dot_product =
    std::transform_reduce(std::par_unseq, x.begin(), x.end(), y.begin(), double(0.0));
```

A parallel word count could be written with binary-binary `transform_reduce` as:

```
bool is_word_beginning(char left, char right)
{
    return std::isspace(left) && !std::isspace(right);
}

std::size_t word_count(std::string_view s)
{
    if (s.empty()) return 0;

    // Count the number of characters that start a new word.
    return std::transform_reduce(
        std::par_unseq,

        // "Left" input sequence:  s[0], s[1], ..., s[s.size() - 2]
        s.begin(), s.end() - 1,
        // "Right" input sequence: s[1], s[2], ..., s[s.size() - 1]
        s.begin() + 1,

        // Initial value for reduction: if the first character
        // is not a space, then it's the beginning of a word.
        std::size_t(!std::isspace(s.front()) ? 1 : 0),

        // Binary transformation operation.
        std::plus<std::size_t>(),

        // Binary transformation operation: Return 1 when we hit
        // a new word.
        is_word_beginning
    );
}
```

§ 3. FAQ

Q: Why does this need to be fixed now?

- `inner_product` currently has ordering guarantees which prevent parallelization. If we fix them in the future, it will have to be a breaking change.
- The order of parameters in some of the new algorithms from the Parallelism TS v1 is inconsistent. If we fix them in the future, it will have to be a breaking change.

Q: The initial value parameter comes before the operation parameters in `exclusive_scan`, but after the operation parameters in `inclusive_scan`. Isn't this also inconsistent?

Initially, I thought this was inconsistent. However, I've since learned from LWG that this was done intentionally. The goal is to keep parameters fixed relative to the other signatures for that algorithm. For example, since `inclusive_scan` does not require an initial value parameter, it would be odd to change from:

```
OutputIt inclusive_scan(InputIt, InputIt, OutputIt);
OutputIt inclusive_scan(InputIt, InputIt, OutputIt, BinaryOperation);
OutputIt inclusive_scan(InputIt, InputIt, OutputIt, BinaryOperation, T init);
```

to this:

```
OutputIt inclusive_scan(InputIt, InputIt, OutputIt);
OutputIt inclusive_scan(InputIt, InputIt, OutputIt, BinaryOperation);
OutputIt inclusive_scan(InputIt, InputIt, OutputIt, T init, BinaryOperation);
```

because it makes the order of `BinaryOperation` inconsistent.

Q: Why does this paper not address the parallel signature of `adjacent_difference`?

`adjacent_difference` uses accumulator style wording and lives in <numeric>, but it is not truly an accumulation, and has none of the associativity and commutativity concerns of the other <numeric> algorithms. SG1 preferred to follow the approach taken with `for_each` and relax the ordering requirement for the parallel signature of `adjacent_difference` instead of adding a new algorithm with weaker guarantees. [P0574r1] contains the necessary changes for `adjacent_difference`.

§ 4. Wording

The following changes are relative to the latest working paper as of 03-02-2017 ([N4640]) with the changes from [P0467r2] applied.

The ❡ character is used to denote a placeholder section number which the editor shall choose.

§ 4.1. Wording for `transform_reduce/inner_product`

Apply the following changes to the declarations of `transform_reduce` in the `<numeric>` header synopsis in 26.8.1 [**numerics.ops.overview**]:

```
template <class InputIterator1, class InputIterator2,
          class T>
  T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                     InputIterator2 first2,
                     T init);
template <class InputIterator1, class InputIterator2,
          class T,
          class BinaryOperation1, class BinaryOperation2>
  T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                     InputIterator2 first2,
                     T init,
                     BinaryOperation1 binary_op1,
                     BinaryOperation2 binary_op2);
template <class InputIterator,
          class UnaryOperation,
          class T,
          class BinaryOperation, class UnaryOperation>
  T transform_reduce(InputIterator first, InputIterator last,
                     UnaryOperation unary_op,
                     T init,
                     BinaryOperation binary_op, UnaryOperation unary_op);
template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
          class T>
  T transform_reduce(ExecutionPolicy&& exec, // see 25.2.5
                     ForwardIterator1 first1, ForwardIterator1 last1,
                     ForwardIterator2 first2,
                     T init);
template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
          class T,
          class BinaryOperation1, class BinaryOperation2>
  T transform_reduce(ExecutionPolicy&& exec, // see 25.2.5
                     ForwardIterator1 first1, ForwardIterator1 last1,
                     ForwardIterator2 first2,
                     T init,
                     BinaryOperation1 binary_op1,
                     BinaryOperation2 binary_op2);
template <class ExecutionPolicy, class ForwardIterator,
          class UnaryOperation,
          class T,
          class BinaryOperation, class UnaryOperation>
  T transform_reduce(ExecutionPolicy&& exec, // see 25.2.5
                     ForwardIterator first, ForwardIterator last,
                     UnaryOperation unary_op,
                     T init,
```

```
                    BinaryOperation binary_op, UnaryOperation unary_op);

template <class InputIterator1, class InputIterator2, class T>
  T inner_product(InputIterator1 first1, InputIterator1 last1,
                  InputIterator2 first2, T init);
template <class InputIterator1, class InputIterator2, class T,
          class BinaryOperation1, class BinaryOperation2>
  T inner_product(InputIterator1 first1, InputIterator1 last1,
                  InputIterator2 first2, T init,
                  BinaryOperation1 binary_op1,
                  BinaryOperation2 binary_op2);
template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
          class T>
  T inner_product(ExecutionPolicy&& exec, // see 25.2.5
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, T init);
template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
          class T, class BinaryOperation1, class BinaryOperation2>
  T inner_product(ExecutionPolicy&& exec, // see 25.2.5
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, T init,
                  BinaryOperation1 binary_op1,
                  BinaryOperation2 binary_op2);
```

Apply the following changes to the definitions of `transform_reduce` in 26.8.4 **[transform.reduce]**:

26.8.4 Transform reduce [transform.reduce]:

```
template <class InputIterator1, class InputIterator2,
          class T>
  T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                     InputIterator2 first2,
                     T init);
template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
          class T>
  T transform_reduce(ExecutionPolicy&& exec,
                     ForwardIterator1 first1, ForwardIterator1 last1,
                     ForwardIterator2 first2,
                     T init);
```

— *Effects:* Equivalent to: `return transform_reduce(first1, last1, first2, init, plus<>(), multiplies<>());`

```
template <class InputIterator1, class InputIterator2,
          class T,
          class BinaryOperation1, class BinaryOperation2>
  T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                     InputIterator2 first2,
                     T init,
                     BinaryOperation1 binary_op1,
                     BinaryOperation2 binary_op2);
template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
          class T,
          class BinaryOperation1, class BinaryOperation2>
  T transform_reduce(ExecutionPolicy&& exec,
                     ForwardIterator1 first1, ForwardIterator1 last1,
                     ForwardIterator2 first2,
                     T init,
                     BinaryOperation1 binary_op1,
                     BinaryOperation2 binary_op2);
```

— *Returns:* `GENERALIZED_SUM(binary_op1, init, binary_op2(*i, *(first2 + (i - first1))), ...)` for every iterator `i` in `[first1, last1)`.

>
> *Requires:* Neither `binary_op1` nor `binary_op2` shall invalidate subranges, or modify elements in the ranges `[first1, last1)` and `[first2, first2 + (last1 - first1))`.
>
> *Complexity:* `O(last1 - first1)` applications each of `binary_op1` and `binary_op2`.

```
template <class InputIterator,
          class UnaryOperation,
          class T,
          class BinaryOperation, class UnaryOperation>
  T transform_reduce(InputIterator first, InputIterator last,
                     UnaryOperation unary_op,
                     T init,
                     BinaryOperation binary_op, UnaryOperation unary_op);
template <class ExecutionPolicy, class ForwardIterator,
          class UnaryOperation,
          class T,
          class BinaryOperation, class UnaryOperation>
  T transform_reduce(ExecutionPolicy&& exec,
                     ForwardIterator first, ForwardIterator last,
                     UnaryOperation unary_op,
                     T init,
                     BinaryOperation binary_op, UnaryOperation unary_op);
```

1 *Returns:* `GENERALIZED_SUM(binary_op, init, unary_op(*i), ...)` for every iterator `i` in `[first, last)`.

2 *Requires:* Neither `unary_op` nor `binary_op` shall invalidate subranges, or modify elements in the range `[first, last)`.

3 *Complexity:* `O(last - first)` applications each of `unary_op` and `binary_op`.

4 *Notes:* `transform_reduce` does not apply `unary_op` to `init`.

Direction to the Editor: Reorder the declarations and definitions of `inner_product` to come before the declarations of `transform_reduce()` in the `<numeric>` header synopsis and class definitions in 26.8 to maintain consistency with `accumulate`/`reduce` and `partial_sum`/`inclusive_scan`.

§ 4.2. Wording for `transform_exclusive_scan`

Apply the following changes to the declarations of `transform_exclusive_scan` in the `<numeric>` header synopsis in 26.8.1 [**numerics.ops.overview**]:

```
template <class InputIterator, class OutputIterator,
          class UnaryOperation,
          class T,
          class BinaryOperation,
          class UnaryOperation>
OutputIterator transform_exclusive_scan(InputIterator first, InputIterator last,
                                        OutputIterator result,
                                        UnaryOperation unary_op,
                                        T init,
                                        BinaryOperation binary_op,
                                        UnaryOperation unary_op);
template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
          class UnaryOperation,
          class T,
          class BinaryOperation,
          class UnaryOperation>
ForwardIterator2 transform_exclusive_scan(ExecutionPolicy&& exec, // see 25.2.5
                                          ForwardIterator1 first, ForwardIterator1 last,
                                          ForwardIterator2 result,
                                          UnaryOperation unary_op,
                                          T init,
                                          BinaryOperation binary_op,
                                          UnaryOperation unary_op);
```

Apply the following changes to the definitions of `transform_exclusive_scan` in 26.8.9 [**transform.exclusive.scan**]:

```
template <class InputIterator, class OutputIterator,
          class UnaryOperation,
          class T,
          class BinaryOperation,
          class UnaryOperation>
OutputIterator transform_exclusive_scan(InputIterator first, InputIterator last,
                                        OutputIterator result,
                                        UnaryOperation unary_op,
                                        T init,
                                        BinaryOperation binary_op,
                                        UnaryOperation unary_op);
template <class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
          class UnaryOperation,
          class T,
          class BinaryOperation,
          class UnaryOperation>
ForwardIterator2 transform_exclusive_scan(ExecutionPolicy&& exec,
                                          ForwardIterator1 first, ForwardIterator1 last,
                                          ForwardIterator2 result,
                                          UnaryOperation unary_op,
                                          T init,
                                          BinaryOperation binary_op,
                                          UnaryOperation unary_op);
```

§ 4.3. Wording for `transform_inclusive_scan`

Apply the following changes to the declarations of `transform_inclusive_scan` in the `<numeric>` header synopsis in 26.8.1 [**numerics.ops.overview**]:

```
template <class InputIterator, class OutputIterator,
          class UnaryOperation,
          class BinaryOperation,
          class UnaryOperation>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                        OutputIterator result,
                                        UnaryOperation unary_op,
                                        BinaryOperation binary_op,
                                        UnaryOperation unary_op);
template <class InputIterator, class OutputIterator,
          class UnaryOperation,
          class BinaryOperation,
          class UnaryOperation,
          class T>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                        OutputIterator result,
                                        UnaryOperation unary_op,
                                        BinaryOperation binary_op,
                                        UnaryOperation unary_op,
                                        T init);
template <class ExecutionPolicy,
          class ForwardIterator1, class ForwardIterator2,
          class UnaryOperation,
          class BinaryOperation,
          class UnaryOperation>
ForwardIterator2 transform_inclusive_scan(ExecutionPolicy&& exec, // see 25.2.5
                                          ForwardIterator1 first, ForwardIterator1 last,
                                          ForwardIterator2 result,
                                          UnaryOperation unary_op,
                                          BinaryOperation binary_op,
                                          UnaryOperation unary_op);
template <class ExecutionPolicy,
          class ForwardIterator1, class ForwardIterator2,
          class UnaryOperation,
          class BinaryOperation,
          class UnaryOperation,
          class T>
ForwardIterator2 transform_inclusive_scan(ExecutionPolicy&& exec, // see 25.2.5
                                          ForwardIterator1 first, ForwardIterator1 last,
                                          ForwardIterator2 result,
                                          UnaryOperation unary_op,
                                          BinaryOperation binary_op,
                                          UnaryOperation unary_op,
                                          T init);
```

Apply the following changes to the definitions of `transform_inclusive_scan` in 26.8.10 [**transform.inclusive.scan**]:

```
template <class InputIterator, class OutputIterator,
          class UnaryOperation,
          class BinaryOperation,
          class UnaryOperation>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                        OutputIterator result,
                                        UnaryOperation unary_op,
                                        BinaryOperation binary_op,
                                        UnaryOperation unary_op);
template <class InputIterator, class OutputIterator,
          class UnaryOperation,
          class BinaryOperation,
          class UnaryOperation,
          class T>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                        OutputIterator result,
                                        UnaryOperation unary_op,
                                        BinaryOperation binary_op,
                                        UnaryOperation unary_op,
                                        T init);
template <class ExecutionPolicy,
          class ForwardIterator1, class ForwardIterator2,
          class UnaryOperation,
          class BinaryOperation,
          class UnaryOperation>
ForwardIterator2 transform_inclusive_scan(ExecutionPolicy&& exec,
                                          ForwardIterator1 first, ForwardIterator1 last,
                                          ForwardIterator2 result,
                                          UnaryOperation unary_op,
                                          BinaryOperation binary_op,
                                          UnaryOperation unary_op);
template <class ExecutionPolicy,
          class ForwardIterator1, class ForwardIterator2,
          class UnaryOperation,
          class BinaryOperation,
          class UnaryOperation,
          class T>
ForwardIterator2 transform_inclusive_scan(ExecutionPolicy&& exec,
                                          ForwardIterator1 first, ForwardIterator1 last,
                                          ForwardIterator2 result,
                                          UnaryOperation unary_op,
                                          BinaryOperation binary_op,
                                          UnaryOperation unary_op,
                                          T init);
```

5. Acknowledgement

Thanks to:

- Agustín Bergé for helping identify this issue.
- Alisdair Meredith for assisting with wording and tackling the related problem with `adjacent_difference`.
- Hartmut Kaiser, JF Bastien, Michael Garland and Jared Hoberock for providing feedback.

References

Informative References

[P0467r2]
Iterator Concerns for Parallel Algorithms. URL: https://wg21.link/p0467r2

[P0574r1]
Algorithm Complexity Constraints and Parallel Overloads. URL: https://wg21.link/p0574r1

[N3960]
Jared Hoberock. Working Draft, Technical Specification for C++ Extensions for Parallelism. 28 February 2014. URL: https://wg21.link/n3960

[N4640]
Richard Smith. Working Draft, Standard for Programming Language C++. 6 February 2017. URL: https://wg21.link/n4640